

Docker Swarm Cluster Deployment & Security Guide

This reference manual details the full deployment steps, verification sequences, and firewall rules explicitly applied to the cluster infrastructure environment.

1. Security Group Inbound Firewall Matrix

The following configuration maps directly to the active firewall permissions allocated inside AWS security group `sg-0d5171578a97c2f13` (`launch-wizard-8`):

Type	Protocol	Port Range	Source	Purpose / Scope
HTTP	TCP	80	0.0.0.0/0	Web traffic routing mesh ingress
All ICMP - IPv4	ICMP	All	0.0.0.0/0	Network ping and routing diagnostics
Custom TCP	TCP	8080 - 8099	0.0.0.0/0	Custom application runtime publishing (e.g., webapp, mobapp)
HTTPS	TCP	443	0.0.0.0/0	Secure TLS public service ingress
Custom TCP	TCP	2377	0.0.0.0/0	Swarm management control plane orchestration communications
SSH	TCP	22	0.0.0.0/0	Remote command-line access management

2. Infrastructure Provisioning (User Data Automation Scripts)

Manager Node Initialization script (`manager1`)

```
#!/bin/bash
hostnamectl set-hostname manager1
apt-get update -y
apt-get install -y curl
curl -fsSL https://get.docker.com | sh
systemctl enable docker
systemctl start docker
echo "Docker installed successfully on manager1"
```

Worker Node 1 script (`worker1`)

```
#!/bin/bash
hostnamectl set-hostname worker1
```

```
apt-get update -y
apt-get install -y curl
curl -fsSL https://get.docker.com | sh
systemctl enable docker
systemctl start docker
echo "Docker installed successfully on worker1"
```

Worker Node 2 script (**worker2**)

```
#!/bin/bash
hostnamectl set-hostname worker2
apt-get update -y
apt-get install -y curl
curl -fsSL https://get.docker.com | sh
systemctl enable docker
systemctl start docker
echo "Docker installed successfully on worker2"
```

3. Host Permission Adjustments & Privileges

Grant standard non-root shell profiles immediate execution access to the active engine daemon UNIX runtime socket layer:

```
sudo usermod -aG docker $USER && newgrp docker
```

*Example context executed on target node host (**worker2**):*

```
ubuntu@worker2:~$ sudo usermod -aG docker ubuntu
ubuntu@worker2:~$ newgrp docker
```

Functional Semantic Alternatives:

- Grant Docker Privileges to the Current User
- Enable Docker Access Without sudo
- Add User to Docker Group and Refresh Session
- Configure Docker Permissions for the Current User

Identity & Group Verification Audits

```
getent passwd
getent group
```

4. Swarm Cluster Lifecycle Orchestration

Control Plane Initialization (Executed on **manager1**)

```
docker swarm init --advertise-addr 172.31.1.226
```

Terminal Verification Log:

```
ubuntu@manager1:~$ docker swarm init --advertise-addr 172.31.1.226
Swarm initialized: current node (z3q4gs72xicap9131t1kcd7vd) is now a manager.

To add a worker to this swarm, run the following command:
    docker swarm join --token
    SWMTKN-1-0sugj0c8dmye00kh8p2rlgbdc5l49sesljiscj75h274vsb4-32o4hsactw2naac7i0edrh2t6
    172.31.1.226:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the
instructions.
```

Node Attachment Steps (Worker Nodes Join Sequence)

```
ubuntu@worker1:~$ docker swarm join --token
SWMTKN-1-0sugj0c8dmye00kh8p2rlgbdc5l49sesljiscj75h274vsb4-32o4hsactw2naac7i0edrh2t6
172.31.1.226:2377
This node joined a swarm as a worker.
```

```
ubuntu@worker2:~$ docker swarm join --token
SWMTKN-1-0sugj0c8dmye00kh8p2rlgbdc5l49sesljiscj75h274vsb4-32o4hsactw2naac7i0edrh2t6
172.31.1.226:2377
This node joined a swarm as a worker.
```

Verifying Cluster Topology Maps

```
ubuntu@manager1:~$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
z3q4gs72xicap9131t1kcd7vd	* manager1	Ready	Active	Leader	29.5.3
h4szrpv8jqtqchzkwtuospguva	worker1	Ready	Active		29.5.3
joo6t3bm3gwnnjqp2kqz1kf66	worker2	Ready	Active		29.5.3

Decommissioning Workers from Cluster

```
ubuntu@worker1:~$ docker swarm leave
Node left the swarm.
```

Topology Operation Summary:

The cluster context initially maps 1 Master Leader unit and 2 operational Worker nodes. Executing the self-imposed leave instruction structural command cleanly disconnects the target host node container ecosystem safely from the orchestration ring.

5. Replicated Service Deployments & Scaling Routines

Service Initialization

```
ubuntu@manager1:~$ docker service create --name webapp -p 8080:80 nginx
ykh10hzotekir2q5f1gaof0
overall progress: 1 out of 1 tasks
1/1: running
verify: Service ykh10hzotekir2q5f1gaof0 converged
```

Status Verification Check

```
ubuntu@manager1:~$ docker service ls
ID                NAME        MODE         REPLICAS  IMAGE          PORTS
ykh10hzotek      webapp      replicated   1/1        nginx:latest   *:8080->80/tcp
```

Deep Structure Metadata Inspection Command

```
docker service inspect webapp && docker service inspect webapp --pretty
```

Task Assignment Location Map Lookup

```
ubuntu@manager1:~$ docker service ps webapp
ID                NAME        IMAGE          NODE         DESIRED STATE  CURRENT STATE
ERROR   PORTS
9sr3p7u33w0k     webapp.1    nginx:latest   worker2      Running        Running 3 minutes ago
```

Declarative Scaling Commands

Scaling Up to 5 Tasks:

```
ubuntu@manager1:~$ docker service scale webapp=5
webapp scaled to 5
overall progress: 5 out of 5 tasks
1/5: running | 2/5: running | 3/5: running | 4/5: running | 5/5: running
verify: Service webapp converged
```

Scaling Down to 3 Tasks:

```
ubuntu@manager1:~$ docker service update --replicas 3 webapp
webapp
```

```
overall progress: 3 out of 3 tasks
1/3: running | 2/3: running | 3/3: running
verify: Service webapp converged
```

Service Destruction and Cleanup Operations

```
ubuntu@manager1:~$ docker service rm webapp
webapp
ubuntu@manager1:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@manager1:~$ docker service ls
ID            NAME      MODE      REPLICAS  IMAGE      PORTS
```

6. Cluster-Wide Balanced Workload Topology Verification

Deploying a 6-replica instance block allocations test across the distributed network structure mapping matrix:

```
ubuntu@manager1:~$ docker service create --name mobapp --replicas 6 -p 8081:80 nginx:alpine
wadoqwruo3bqojeh0kflm4u6c
overall progress: 6 out of 6 tasks
1/6: running | 2/6: running | 3/6: running | 4/6: running | 5/6: running | 6/6: running
verify: Service wadoqwruo3bqojeh0kflm4u6c converged
```

Distributed Runtime Container Inspection Mapping (`docker ps`)

Manager Target Context (`manager1`)

CONTAINER ID	IMAGE	NAMES
c788bff2ce85	nginx:alpine	mobapp.5.pzoag5cjiiy6m87284h43aaql
833140ce7350	nginx:alpine	mobapp.2.0w2azstlyccjov01p6mk96zu6

Worker 1 Target Context (`worker1`)

CONTAINER ID	IMAGE	NAMES
d5fb9c932991	nginx:alpine	mobapp.4.ympm48vln6u40r4lnmy3cbi8a
c203005ee7d6	nginx:alpine	mobapp.1.lykpjirujsa9d1ufprmjfhygw

Worker 2 Target Context (`worker2`)

CONTAINER ID	IMAGE	NAMES
13dc22b9cd25	nginx:alpine	mobapp.3.4q1gpjcyfyzxc7p49li5almri
23af4d8013ff	nginx:alpine	mobapp.6.vedm8i974m0wjyzi2bf0pqnbt

Dynamic Scaling Reduction Task

```
ubuntu@manager1:~$ docker service scale mobapp=2
mobapp scaled to 2
```

```
overall progress: 2 out of 2 tasks  
1/2: running | 2/2: running  
verify: Service mobapp converged
```